

Galecto: Technical Deep-Dive & Feature Mechanics

A Detailed Guide for System Troubleshooting and Architecture Mastery

1. Auth & Onboarding Flow (The SaaS Bootstrap)

The onboarding process begins in the `auth-service` using a multi-repository pattern. When a user signs up, the system executes a transactional flow: it creates an **Organization** in PostgreSQL, bootstraps a **Default Project**, and generates a unique **'gl_live' API Key**. This ensures that the frontend can immediately display integration snippets and the `api-gateway` can begin validating incoming telemetry without manual configuration.

2. Telemetry Ingestion Pipeline (The Data Highway)

The Ingestion pipeline is built for high-throughput and low latency using a producer-consumer model. The `api-gateway` acts as the edge node, validating API keys and injecting `traceId` and `spanId` into incoming HTTP headers if they are missing. It then publishes these events to a **Kafka Topic** named `events`, decoupling the ingestion speed from the database write speed and ensuring that traffic spikes don't overwhelm the system.

3. Recursive Trace Stitching (The Intelligence Engine)

Trace stitching is performed dynamically by the `query-service` rather than being pre-calculated at ingestion. When a user requests a trace graph, the service executes a recursive SQL query against the **ClickHouse `events` table** to find all spans sharing the same `traceId`. It then builds a hierarchical tree structure by matching `parentSpanId` to `spanId`, which is then delivered to the React Flow frontend for visual rendering.

4. Shadow Replay Mechanism (The Debugging Proxy)

The Replay System works by re-firing historical requests through a dedicated proxy logic in the `api-gateway`. The system retrieves the original request body and headers (including authorization) from the ClickHouse long-term storage. It then executes a new axios-based request with a special `x-galecto-replay` header, allowing the developer to compare the new result against the original failure in a side-by-side UI view.

5. OLAP Log Searching (The Big Data Explorer)

Log searching is optimized via **ClickHouse's Columnar Storage**, which allows the `query-service` to scan millions of rows in milliseconds. Unlike traditional row-based databases (like PostgreSQL), ClickHouse only reads the columns requested (e.g., `message`, `timestamp`, `level`), which significantly reduces I/O. The search interface supports raw text matching and structured filtering by Service Name or Trace ID to provide instant visibility into distributed logs.

6. Anomaly Detection & Alerting (The Watchman)

The `alert-service` operates as a dedicated Kafka consumer that scans every incoming event for error patterns. It specifically looks for HTTP status codes ≥ 400 and latency values that exceed a pre-defined threshold (e.g., P99 spikes). When an anomaly is detected, it persists an "Incident" in the database and triggers a real-time event that updates the **Alerts Dashboard**, allowing for immediate human intervention and resolution tracking.

7. Real-time Metrics Aggregation (The Pulse)

System-wide metrics are generated using **ClickHouse Aggregate Functions** like `quantile(0.99)(latency)`. The `query-service` periodically runs these intensive calculations to provide the "Quantum Stream" of P99 latencies and error rates seen on the Monitoring page. This approach moves the computational heavy-lifting to the database layer, ensuring the API Gateway remains responsive even under heavy analytic load.

8. Automated Data Retention (The Pruning Worker)

To manage storage costs and performance, the `log-service` runs a background **Retention Worker** using `node-cron`. This worker queries the organization's settings to determine their retention window (e.g., 30 days) and executes an `ALTER TABLE ... DELETE` mutation on ClickHouse. This ensures the cluster remains lean by automatically purging aged telemetry data that is no longer required for active debugging or compliance.

Galecto v1.0.0

Observability without limits.